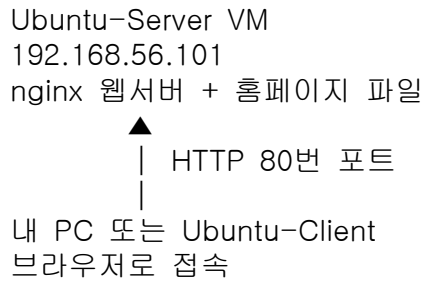


<<홈페이지 만들어보기>>

Ubuntu-Server에 간단한 홈페이지 파일을 만들고, nginx 웹서버를 통해 Client 브라우저에서 보는 실습으로 구조는 다음과 같습니다.



1. 서버에 nginx 설치

Ubuntu-Server에서 실행합니다.

```
sudo apt update
```

```
sudo apt install nginx -y
```

nginx 실행:

```
sudo systemctl start nginx
```

```
sudo systemctl enable nginx
```

상태 확인:

```
sudo systemctl status nginx
```

active (running)이면 정상입니다.

2. 서버 IP 확인

Ubuntu-Server에서 실행합니다.

```
hostname -I
```

예시:

```
10.0.2.15 192.168.56.101
```

브라우저에서 접속할 때는 보통 **호스트 전용 어댑터 IP**인 192.168.56.x를 사용합니다.

예시 서버 IP:

```
192.168.56.101
```

3. 기본 nginx 페이지 확인

내 PC 또는 Ubuntu-Client 브라우저에서 접속합니다.

```
http://192.168.56.101
```

nginx 기본 페이지가 보이면 웹서버 접속은 성공입니다.

4. 홈페이지 파일 만들기

nginx의 기본 웹 문서 위치는 보통 다음 경로입니다.

```
/var/www/html
```

기본 페이지 파일을 백업합니다.

```
sudo cp /var/www/html/index.nginx-debian.html
```

```
/var/www/html/index.nginx-debian.html.bak
```

새 홈페이지 파일을 만듭니다.

```
sudo nano /var/www/html/index.html
```

아래 내용을 붙여넣습니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>나의 첫 번째 웹 서버</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f2f6ff;
      text-align: center;
      padding-top: 80px;
    }

    .box {
      background-color: white;
      width: 600px;
      margin: auto;
      padding: 40px;
      border-radius: 12px;
      box-shadow: 0 0 12px rgba(0,0,0,0.15);
    }

    h1 {
      color: #2563eb;
    }

    p {
      font-size: 18px;
    }

    .info {
      margin-top: 30px;
      color: #555;
      font-size: 14px;
    }
  </style>
</head>
<body>
  <div class="box">
    <h1>Ubuntu 웹 서버 접속 성공!</h1>
    <p>이 페이지는 Ubuntu-Server VM의 nginx 웹서버에서 제공하고 있습니다.</p>
    <p>클라이언트 브라우저에서 서버의 홈페이지를 보고 있습니다.</p>

    <div class="info">
      Server: Ubuntu + nginx<br>
      Port: 80<br>
      Protocol: HTTP
    </div>
  </div>
</body>
</html>

```

저장:

Ctrl + O → Enter → Ctrl + X

5. nginx 재시작

홈페이지 파일을 만든 후 nginx를 재시작합니다.
`sudo systemctl restart nginx`

상태 확인:

```
sudo systemctl status nginx
```

6. 브라우저에서 확인

내 PC 또는 Ubuntu-Client 브라우저에서 다시 접속합니다.
`http://192.168.56.101`

방금 만든 홈페이지가 보이면 성공입니다.
브라우저가 예전 페이지를 보여주면 새로고침합니다.
`Ctrl + F5`

7. 방화벽 확인

웹페이지가 보이지 않으면 서버에서 80번 포트를 허용합니다.
`sudo ufw allow 80/tcp`
`sudo ufw status`

ufw가 꺼져 있다면 켜도 됩니다.
`sudo ufw enable`

다만 SSH로 접속 중이라면 먼저 SSH도 허용해야 합니다.
`sudo ufw allow ssh`
`sudo ufw enable`

8. 접속 테스트 명령어

브라우저 대신 터미널에서도 확인할 수 있습니다.
Client VM에서:
`curl http://192.168.56.101`

HTML 코드가 출력되면 웹서버가 정상 응답하는 것입니다.
서버에서 80번 포트가 열려 있는지 확인:
`sudo ss -tulnp | grep :80`

정상 예시:

```
LISTEN 0 511 0.0.0.0:80
```

9. 실습 흐름 정리

1. Ubuntu-Server에 nginx 설치
2. 서버 IP 확인
3. 클라이언트 브라우저에서 기본 페이지 접속
4. `/var/www/html/index.html` 생성
5. HTML 코드 작성
6. nginx 재시작
7. 브라우저에서 `http://서버IP` 접속
8. 방화벽 80번 포트 확인

10. 핵심 개념

웹서버: nginx
웹 문서 위치: `/var/www/html`
기본 홈페이지 파일: `index.html`
HTTP 포트: 80
클라이언트: 브라우저
서버: Ubuntu-Server VM

즉, 클라이언트가 브라우저에서 다음 주소로 요청하면:
`http://192.168.56.101`

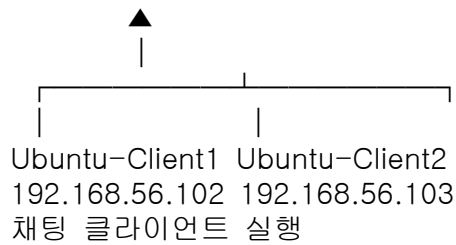
서버의 nginx가 다음 파일을 읽어서 응답합니다.
`/var/www/html/index.html`

<<채팅 네트워크 구축>>

Ubuntu-Server VM

192.168.56.101

채팅 서버 실행



목표 : 서버 1대가 중간에서 메시지를 받아서 두 클라이언트에게 전달하는 채팅 네트워크를 만드는 것입니다.

1. VM 구성

필요한 VM

기존 Ubuntu VM을 복제해서 총 3대를 만듭니다.

Ubuntu-Server

Ubuntu-Client1

Ubuntu-Client2

VirtualBox에서 기존 Ubuntu VM 우클릭 후:

복제 → 완전한 복제 → MAC 주소 새로 생성

각 VM의 네트워크는 동일하게 설정합니다.

어댑터 1: NAT

어댑터 2: 호스트 전용 어댑터

2. IP 확인

각 VM에서 실행합니다.

hostname -I

예시 IP를 다음처럼 정리합니다.

역할	VM 이름	IP 예시
서버	Ubuntu-Server	192.168.56.101
클라이언트 1	Ubuntu-Client1	192.168.56.102
클라이언트 2	Ubuntu-Client2	192.168.56.103

앞으로 예시는 서버 IP를 192.168.56.101로 사용하겠습니다.

3. 통신 확인

Client1에서 Server로 ping

Client1에서 실행:

ping 192.168.56.101

Client2에서 Server로 ping

Client2에서 실행:

ping 192.168.56.101

정상이라면 응답이 와야 합니다.

중지:

Ctrl + C

4. 채팅 서버 만들기

Ubuntu-Server에서 작업합니다.

4-1. 작업 폴더 생성

```
mkdir chat-lab  
cd chat-lab
```

4-2. 서버 파일 생성

```
nano chat_server.py
```

아래 코드를 붙여넣습니다.

```
-----  
import socket  
import threading  
  
HOST = "0.0.0.0"  
PORT = 5000  
  
clients = []  
  
def broadcast(message, sender_socket):  
    for client in clients:  
        if client != sender_socket:  
            try:  
                client.send(message)  
            except:  
                clients.remove(client)  
  
def handle_client(client_socket, client_address):  
    print(f"[접속] {client_address} 접속함")  
  
    while True:  
        try:  
            message = client_socket.recv(1024)  
  
            if not message:  
                break  
  
            print(f"[메시지] {client_address}: {message.decode()}")  
            broadcast(message, client_socket)  
  
        except:  
            break  
  
    print(f"[종료] {client_address} 접속 종료")  
    clients.remove(client_socket)  
    client_socket.close()  
  
def start_server():  
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
    server_socket.bind((HOST, PORT))  
    server_socket.listen()  
  
    print(f"[서버 시작] {HOST}:{PORT}")  
    print("[대기 중] 클라이언트 접속을 기다립니다...")  
  
    while True:
```

```

        client_socket, client_address = server_socket.accept()
        clients.append(client_socket)

        thread = threading.Thread(
            target=handle_client,
            args=(client_socket, client_address)
        )
        thread.start()

start_server()
-----

```

저장:

Ctrl + O → Enter → Ctrl + X

5. 채팅 클라이언트 만들기

Client1과 Client2 둘 다 같은 파일을 만듭니다.

5-1. 작업 폴더 생성

Client1에서:

```

mkdir chat-lab
cd chat-lab

```

Client2에서도:

```

mkdir chat-lab
cd chat-lab

```

5-2. 클라이언트 파일 생성

Client1, Client2 둘 다 실행:

```

nano chat_client.py

```

아래 코드를 붙여넣습니다.

```

-----
import socket
import threading

SERVER_IP = "192.168.56.101"
PORT = 5000

nickname = input("닉네임 입력: ")

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((SERVER_IP, PORT))

def receive_message():
    while True:
        try:
            message = client_socket.recv(1024).decode()
            if message:
                print("Wn" + message)
            else:
                break
        except:
            print("서버와 연결이 끊어졌습니다.")
            break

def send_message():

```

```

while True:
    message = input()

    if message.lower() == "/quit":
        client_socket.close()
        break

    full_message = f"{nickname}: {message}"
    client_socket.send(full_message.encode())

receive_thread = threading.Thread(target=receive_message)
receive_thread.daemon = True
receive_thread.start()

send_message()
-----

```

중요한 부분:

SERVER_IP = "192.168.56.101"

여기에는 실제 서버 VM의 IP를 넣어야 합니다.

6. 방화벽 설정

Ubuntu-Server에서 실행합니다.

```

sudo ufw allow 5000/tcp
sudo ufw status

```

ufw가 꺼져 있다면:

```

sudo ufw enable

```

다시 확인:

```

sudo ufw status

```

다음과 비슷하게 보여야 합니다.

```

5000/tcp ALLOW Anywhere

```

7. 채팅 실행 순서

반드시 서버 먼저 실행해야 합니다.

7-1. Server에서 채팅 서버 실행

Ubuntu-Server에서:

```

cd chat-lab
python3 chat_server.py

```

정상 실행 예시:

```

[서버 시작] 0.0.0.0:5000

```

```

[대기 중] 클라이언트 접속을 기다립니다...

```

7-2. Client1에서 접속

Ubuntu-Client1에서:

```

cd chat-lab
python3 chat_client.py

```

닉네임 입력:

```

닉네임 입력: client1

```


7-3. Client2에서 접속

Ubuntu-Client2에서:

```
cd chat-lab
```

```
python3 chat_client.py
```

닉네임 입력:

닉네임 입력: client2

8. 채팅 테스트

Client1에서 입력:

안녕하세요. 저는 client1입니다.

Client2 화면에 다음처럼 보여야 합니다.

client1: 안녕하세요. 저는 client1입니다.

Client2에서 입력:

반갑습니다. 저는 client2입니다.

Client1 화면에 다음처럼 보여야 합니다.

client2: 반갑습니다. 저는 client2입니다.

9. 수업용 개념 정리

서버 역할

Ubuntu-Server는 채팅 서버입니다.

클라이언트들이 접속할 수 있도록 5000번 포트를 열고 기다립니다.

클라이언트 역할

Ubuntu-Client1과 Ubuntu-Client2는 채팅 클라이언트입니다.

서버 IP와 포트 번호를 이용해서 서버에 접속합니다.

포트 역할

5000번 포트는 이번 실습에서 채팅 서비스용 포트입니다.

데이터 흐름

Client1 → Server → Client2

Client2 → Server → Client1

클라이언트끼리 직접 메시지를 주고받는 것이 아니라, 서버를 거쳐서 메시지가 전달됩니다.

10. 오류 점검표

1) ping이 안 될 때

각 VM에서 IP 확인:

```
hostname -I
```

VirtualBox 설정 확인:

어댑터 2: 호스트 전용 어댑터

2) 클라이언트 접속이 안 될 때

서버에서 채팅 서버가 실행 중인지 확인:

```
python3 chat_server.py
```

서버에서 5000번 포트가 열려 있는지 확인:

```
sudo ss -tulnp | grep 5000
```

정상 예시:

```
LISTEN 0 128 0.0.0.0:5000
```

3) 방화벽 때문에 안 될 때
서버에서 확인:
`sudo ufw status`

5000번 포트 허용:
`sudo ufw allow 5000/tcp`

4) 서버 IP를 잘못 입력했을 때
Client1, Client2의 `chat_client.py`에서 이 부분을 확인합니다.
`SERVER_IP = "192.168.56.101"`

서버 VM의 실제 IP와 같아야 합니다.

[최종 실습 흐름]

1. Ubuntu VM 3대 준비
 - Ubuntu-Server
 - Ubuntu-Client1
 - Ubuntu-Client2
2. 각 VM 네트워크 설정
 - NAT
 - 호스트 전용 어댑터
3. 각 VM IP 확인
 - `hostname -I`
4. 클라이언트에서 서버 ping 테스트
 - `ping 서버IP`
5. 서버에서 채팅 서버 실행
 - `python3 chat_server.py`
6. 서버 방화벽에서 5000번 포트 허용
 - `sudo ufw allow 5000/tcp`
7. Client1, Client2에서 채팅 클라이언트 실행
 - `python3 chat_client.py`
8. 서로 메시지 주고받기